ASP.NET Core will automatically deserialize the request for you from JSON, creating the `Product` argument.

Listing 7.3 Automatically deserializing a JSON request from the body

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.MapPost("/product", (Product product) => $"Received {product}");  ❶

app.Run();

record Product(int Id, string Name, int Stock);                       ❷
```

❶ Product is a complex type, so it's bound to the JSON body of the request.

❷ Product doesn't implement TryParse, so it's a complex type.

If you send a `POST` request to `/product` for the app in listing 7.3, you need to provide valid JSON in the request body, such as

```
{ "id": 1, "Name": "Shoes", "Stock": 12 }
```

ASP.NET Core uses the built-in System.Text.Json library to deserialize the JSON into a `Product` instance and uses it as the `product` argument in the handler.

Configuring JSON binding with System.Text.Json

The System.Text.Json library, introduced in .NET Core 3.0, provides a high- performance, low-allocation JSON serialization library. It was designed to be something of a successor to the ubiquitous Newtonsoft.Json library, but it trades flexibility for performance.

Minimal APIs use System.Text.Json for both JSON deserialization (when binding to a request's body) and serialization (when writing results, as you saw in chapter 6). Unlike for MVC and